ItinerYey — Hackathon Submission Draft

> Deadline: **10:00 AM, October 10. No extensions.** Fill in every `[TODO]` before submitting.

📁 The Project

Project name: ItinerYey

Short description:

> ItinerYey is a 100% offline-capable commute assistant for Filipino commuters. It uses a local Taglish AI and a custom A* route planner to give jeepney, bus, LRT, MRT, UV Express, and tricycle directions — with zero data sent to the cloud. Commuters can also scan a jeepney signboard with their camera to instantly identify if it passes their destination.

Team members: `[TODO: Add full names of all team members]`

Public GitHub repository: `[TODO: Push to GitHub and paste the URL here]`

> CAUTION
>
> Make sure the repo is set to **Public** before submitting. Run `git push` now if you haven't.

🎬 The Proof

Demo video: `[TODO: Record a 3–5 minute screen recording. Script below ↓]`

X / LinkedIn video URL: `[TODO: Upload video to X or LinkedIn and paste the post URL]`

What runs locally (on-device, zero cloud)

- **Local LLM** — `qwen2.5:3b` via Ollama: understands Taglish questions, extracts origin & destination
- **Local Vision model** — `qwen2.5vl:3b` via Ollama: reads jeepney signboard text from photos
- **A* route planner** — custom pathfinding engine across 47,239 transit segments

- **Offline route index** — compressed local snapshot of 1,719 GTFS routes (`data/offline/snapshot.json.gz`)
- **Place resolver** — fuzzy stop-name matching against the full offline stop directory

What requires internet

- **Supabase** (PostgreSQL) — stores crowd-sourced community routes and place aliases. The app falls back to the local snapshot automatically if offline.
- Initial page load (Next.js app served from `localhost`)

> NOTE
>
> During the demo, turning off WiFi still allows full route planning — the A* engine and AI run entirely from the local snapshot and Ollama.

📋 The Disclosures

Models used

Model	Purpose	Runs where
`qwen2.5:3b`	Taglish NLU — extract origin/destination from free-text questions	Locally via Ollama
`qwen2.5vl:3b`	Vision — read jeepney signboard text from photos	Locally via Ollama

Technologies and frameworks

Technology	Role
Next.js 14 (App Router)	Full-stack web framework
React 18	UI component library
TypeScript	Language
Tailwind CSS	Styling
Leaflet / React-Leaflet	Interactive route map
Supabase (PostgreSQL)	Cloud database for community routes
Ollama	Local LLM inference runtime
Custom A* engine	Multi-modal transit pathfinding
GTFS (Sakay.ph)	Philippine transit data standard

APIs and cloud services

Service	What it's used for	Is it required?
Supabase	Store/read community-submitted routes and place aliases	No — app falls back to offline snapshot
Sakay.ph GTFS	Source of 1,719 PH transit routes (one-time seed, not live)	No — already seeded locally

> No OpenAI, Gemini, Anthropic, or any paid AI API is used. All inference is local.

Existing code and assets

- **GTFS data** — Philippine transit routes from Sakay.ph open dataset (public, freely available)
- **Leaflet** — open-source map library (BSD-2)
- **Supabase JS client** — official open-source SDK

AI development tools

- Google Antigravity (AI pair programming assistant) — used to help write and debug code during the hackathon

🧠 Key Question: Why does this product benefit from running AI locally?

> Filipino commuters face a real connectivity problem: jeepney routes in the Philippines are not on Google Maps, and many commuters travel through areas with poor or expensive mobile data. ItinerYey benefits from local AI in three specific ways:
>
> **1. It works with no signal.** The entire A* route planner and the 47,239-segment transit graph run from a compressed on-device snapshot. A commuter on a moving jeepney with zero bars can still get directions.
>
> **2. Taglish understanding without cloud latency.** The local `qwen2.5:3b` model understands natural Filipino speech patterns — mixing Tagalog and English mid-sentence — without sending any personal location data to a server. Privacy is preserved by design.
>
> **3. Real-time signboard scanning on-device.** The vision model (`qwen2.5vl:3b`) reads jeepney signboard text through the camera and instantly checks if that jeepney passes the commuter's destination. This works offline because the full route index is local — no API call needed to cross-reference the signboard text against 1,719 routes.
>
> Running AI locally is not a technical choice — it is the product. A cloud-dependent commute assistant fails exactly when commuters need it most: underground, in tunnels, or in areas with congested networks.

📱 Real-World Deployment Roadmap (Mobile-Only Commuters & Hardware)

Judges often ask: "*How will real commuters use this if they only have their smartphone?*"

- **1. Native On-Device Mobile App (Consumer Launch):** The open-weight models (`Qwen2.5-1.5B` / `Gemma-2B`) will be quantized and embedded into an iOS & Android app using **ExecuTorch (Meta)** or **MediaPipe / LiteRT (Google)**. The phone's built-in NPU (Apple Neural Engine, Snapdragon NPU) executes the AI directly in local memory with 0 internet connection and no external hardware.
- **2. Terminal Edge Kiosks (Transit Infrastructure):** At major crowded terminals (Cubao, Starmall Alabang, SM North), ItinerYey can run on a $35 edge device (Raspberry Pi / Orange Pi)

broadcasting a local captive portal Wi-Fi. Commuters connect without data/load to plan their trips or scan signboards on their phone's browser.

- **3. Current Hackathon Prototype:** Demonstrates the end-to-end local architecture: an edge-server instance running Next.js + Ollama, accessible simultaneously by laptops and mobile phones over local Wi-Fi / hotspot with zero cloud calls.

🎬 Demo Video Script (3–5 mins)

Use this as your recording guide:

0:00 — Hook (30 sec)

> *"Millions of Filipinos ride jeepneys every day, but there's no Google Maps for jeepneys. You have to ask strangers or just guess. We built ItinerYey — a commute AI that works with no internet, understands Taglish, and runs entirely on your laptop."*

0:30 — Show the status bar (15 sec)

Point to the green pills: **Online — not needed · Local LLM · Local vision · 1,719 on-device routes**

> *"Everything you see here runs on this machine. Zero bytes to the cloud."*

0:45 — Ask AI demo (90 sec)

Click the suggestion: **"Galing Recto papuntang Katipunan, magkano?"** While it thinks, say:

> *"The AI is extracting the origin and destination in Taglish, running the A* planner across 47,000 segments, then writing the directions — all on-device."* Show the result: legs, fares, transfer steps.

2:15 — Turn off WiFi (30 sec)

> *"Watch — I disconnect from the internet."* Ask another question. Show it still works. *"Still working. No cloud required."*

2:45 — Scan Signboard demo (60 sec)

Switch to **Scan signboard** tab. Click **Try sample signboard**.

> *"A commuter sees a jeepney but doesn't know if it goes where they're going. They point their camera at the signboard. The vision AI reads it on-device and checks our route database."* Show the result: route match, fare, whether it passes the destination.

3:45 — Crowd-sourced data (30 sec)

Click **Explore** in the nav. Show the 1,719 routes.

> *"The route database is crowd-sourced. Commuters submit routes they know. Each one gets a community confidence score. The more people verify it, the more reliable it becomes."*

4:15 — Closing (15 sec)

> *"ItinerYey — because the best commute assistant is the one that works when you have no signal."*

✅ Final Pre-Submission Checklist

- Push latest code to **public** GitHub repo
- Record demo video (follow script above)
- Upload video to X or LinkedIn
- Fill in team member names above
- Paste GitHub URL above
- Paste video URL above
- Run the place_aliases SQL in Supabase dashboard
- Make sure Ollama is running: `ollama serve`
- Test Ask AI one more time before presenting
- Practice the pitch once out loud